

Deltafold — Documentation technique exhaustive du pipeline contrastif

Création du jeu de données, architecture, objectif (MoCo + sous-structures), entraînement, métriques et résultats

Document préparatoire — réunion superviseur

June 25, 2026

Abstract

Ce document décrit, au niveau du code source, l'intégralité du pipeline contrastif de Deltafold : apprentissage non supervisé de représentations de structures protéiques (virome) sur Apple Silicon (MPS, 16 Go). Il couvre (i) la construction de la sous-base corrigée et le split train/val, (ii) les deux familles d'encodeurs hiérarchiques (invariant asymétrique et équivariant GVP/GCP), (iii) l'objectif courant — MoCo-v2 avec paires positives par sous-structures (Hermosilla & Ropinski) — ainsi que les objectifs hérités (InfoNCE, SupCon, soft-SupCon), (iv) la boucle d'entraînement et son ingénierie mémoire, (v) le protocole d'évaluation et (vi) les résultats réels mesurés. Une dernière section liste sans complaisance les angles morts et les questions que le superviseur posera probablement.

Le constat central, à assumer dès le début de la réunion : une fois la fuite de TM-score retirée des données, *aucun run entraîné ne dépasse franchement la baseline « features brutes + init. aléatoire »* sur la métrique cible (TM-rho). C'est le fait le plus important du projet à date ; le reste du document sert à argumenter *pourquoi* et à séparer un problème de pipeline d'un problème d'expressivité du modèle.

1 Vue d'ensemble

Deltafold apprend un plongement (embedding) par protéine tel que la proximité dans l'espace latent reflète la proximité structurale (mesurée *a posteriori* par TM-score et par le clustering Foldseek). Chaque structure (PDB prédite, virome) est « liftée » en un **complexe combinatoire de protéine** (PCC) à quatre rangs (résidus, contacts, structures secondaires, global). Un encodeur à passage de messages hiérarchique produit un vecteur de lecture $h \in \mathbb{R}^{128}$, entraîné par objectif contrastif. L'évaluation compare le clustering/ordonnancement induit par les embeddings aux clusters Foldseek et aux TM-scores.

Fichiers principaux : préparation des données `scripts/utilities/topotein_lifter.py`, `scripts/utilities/bui` ; split `train.py` ; architectures `asymmetric_topotein.py`, `equivariant_topotein.py`, `rbf.py` ; objectif `moco.py`, `substructure.py`, `contrastive_engine.py` ; boucle `train_contrastive.py`, `contrastive_data.py` ; évaluation `scripts/analysis/epoch_eval.py`.

2 Création du jeu de données

2.1 Source et featurisation (le « lifting »)

Les structures brutes proviennent d'une archive `virome_pdb.zip` (jeu « hoan »). `topotein_lifter.py` convertit chaque PDB en un dictionnaire `.pt` à quatre rangs. Toute la géométrie dérive des coordonnées $C\alpha$; les invariants de séquence/structure sont calculés une fois et stockés.

- **Rang 0 — résidus (rank0)**, largeur de features fixe = 64 : acides aminés one-hot (alphabet 23, aa) + alphabet structural **3Di de Foldseek** one-hot (21, 3di, obtenu en appelant le binaire `foldseek createdb/convert2fasta`) + dièdres ϕ/ψ encodés en $(\sin \phi, \cos \phi, \sin \psi, \cos \psi)$ (4,

`phi_psi`) + encodage positionnel sinusoïdal (16, `positional_encoding`). Les `ca_coords` sont aussi stockées (géométrie).

- **Rang 1 — contacts (rank1)** : graphe des $k = 16$ plus proches voisins sur les $C\alpha$ (auto-boucle retirée). Pour chaque arête : `distance` (scalaire $C\alpha-C\alpha$), `distance_encoding` (sinusoïdal 16-d), et `vector` = $c_j - c_i$ (déplacement 3D, $(N, 16, 3)$). Le vecteur n'est consommé que par l'encodeur *equivariant*.
- **Rang 2 — structures secondaires (SSE) (rank2)** : segmentation contiguë issue de `biotite.struc.annotate` (type DSSP-like Hélice/Feuillet/Coil). Chaque SSE porte 12 features = type one-hot (4) + descripteurs de forme PCA (5) + valeurs propres (3), calculés par ACP sur les $C\alpha$ membres.
- **Rang 3 — global (rank3)**, 10 features : taille de la protéine, rayon de giration, descripteurs de forme globaux (5) et valeurs propres globales (3).

Choix de configuration importants, encodés dès le lifting (`TopoteinLifter`) : par défaut `lift_residue=False` et `lift_positional=False` — l'identité du résidu et l'encodage positionnel sont stockés comme *zéros*. C'est l'équivalent bit-à-bit des drapeaux d'entraînement `-no-residue -no-positional` (cf. §3.3), tout en gardant la largeur 64 fixe. Autrement dit : *le modèle « de référence » apprend sans identité de séquence ni position*, pour le forcer à utiliser la topologie 3D plutôt que des raccourcis de composition/séquence.

2.2 Filtre qualité pLDDT

Une liste blanche d'accessions à pLDDT moyen ≥ 70 (`data/subdataset_files.txt`) sert de barrière qualité, orthogonale aux corrections de biais. Le pLDDT est lu dans le facteur B des lignes `ATOM` des PDB. Sur $\sim 31,7k$ structures featurisées, 31 763 passent ce filtre.

2.3 L'ancien downsampler et ses quatre biais (à ne pas reproduire)

`filter_dataset.py` (maintenant marqué *DEPRECATED*) produisait `subdataset_files_refined.txt` (3 647 protéines) avec quatre biais documentés :

1. **Singletons supprimés** — seuls les clusters multi-membres étaient gardés : le modèle ne voyait jamais le cas « pas de voisin structural ».
2. **Distribution aplatie** — « max 2 par cluster » détruisait la longue traîne naturelle des tailles de cluster.
3. **Folds choisis à la main** — une liste `target_keywords` (LigT, UL43, I3L, SSB...) injectait de force certaines protéines.
4. **Intrication sélection $\Leftrightarrow$ évaluation** — ces mêmes protéines-mots-clés étaient les cas de test aval : fuite de l'ensemble d'évaluation dans la sélection.

2.4 La sous-base corrigée (pipeline actuel)

`build_corrected_subbase.py` corrige les quatre biais. Pipeline déterministe (`-seed 42`, cible `-target 4000`, `-val-ratio 0.2`) :

1. **Liste blanche pLDDT** appliquée au pool des `.pt` : 31 763 accessions disponibles.
2. **Contrôles mots-clés mis de côté** (correction #3/#4) : toute protéine dont le nom contient un mot-clé (LigT, UL43, ENT4, I3L, SSB, phosphodiesterase, OB-fold) part dans un manifeste *séparé* `_controls.txt` (36 protéines), exclu de train/val.
3. **Déduplication par séquence exacte** (correction #4, *non optionnelle*). Empreinte = `blake2b(longueur, arg r`. Des séquences identiques sont dispersées dans des clusters Foldseek *différents* ; sans dedup, le split « cold cluster » fuirait quand même des doublons exacts (vérifié : $\sim 667/808$ protéines de val

étaient des copies exactes de train avant correction). Effet : $31\,727 \rightarrow 18\,728$ structures uniques (**12 999 copies redondantes supprimées, $\sim 32\%$ du pool**).

4. **Regroupement en clusters Foldseek** (`cluster.tsv`) ; les accessions absentes sont leurs propres singletons. Pool : 18 728 protéines dans 2 247 clusters (680 singletons, 1 567 multi-membres).
5. **Split « cold cluster »** (correction #4) : clusters entiers assignés à train *ou* val (aucun cluster ne traverse le split). Accumulation de clusters mélangés jusqu'à $\sim 20\%$ du pool en val.
6. **Sous-échantillonnage proportionnel intra-cluster** (corrections #1/#2) : on garde une fraction $f = \text{target}/\text{pool} = 0.214$ des membres de *chaque* cluster (arrondi stochastique, espérance exacte). Les singletons sont gardés avec probabilité f , la longue traîne est mise à l'échelle par le même facteur — la distribution des tailles et le déséquilibre sont préservés (cf. Fig. 1), contrairement au « max 2 ».
7. **Vérification d'absence de fuite** : assertions que 0 cluster traverse le split et 0 séquence de val apparaît dans train.

Table 1: Sous-base corrigée — chiffres effectifs (`subbase_corrected_stats.json`).

Disponibles après pLDDT	31 763	Pool déduplicué	18 728
Contrôles mots-clés (hors train/val)	36	Clusters du pool	2 247
Copies redondantes supprimées	12 999	dont singletons	680
Taux d'échantillonnage f	0.214	Plus gros cluster	828
Train	3 191 (923 cl.)	Validation	813 (205 cl.)
Fuites de séquence train $\leftrightarrow$ val	0	Clusters partagés	0
Frac. protéines-singletons (pop.)	0.0363	(sélection)	0.0400

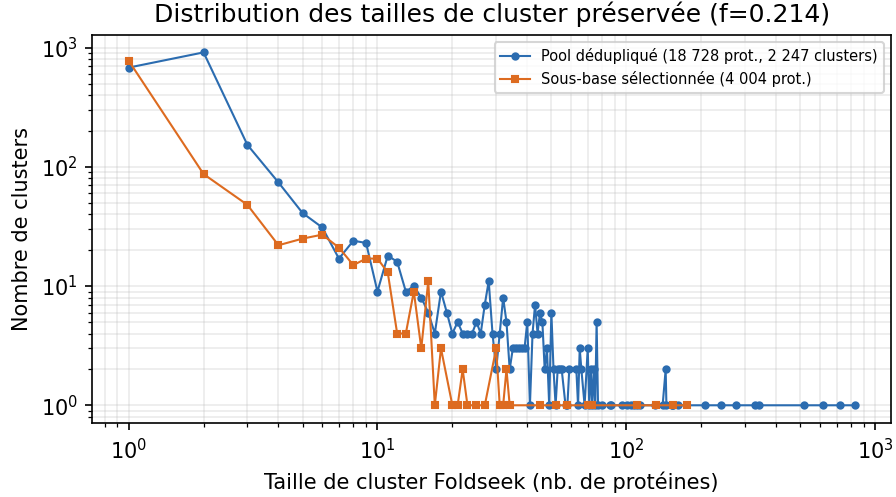


Figure 1: La distribution log-log des tailles de cluster de la sous-base sélectionnée suit celle du pool déduplicué : l'échantillonnage proportionnel ($f = 0.214$) préserve la longue traîne, au lieu de l'aplatir.

À mentionner au superviseur (honnêteté méthodologique). (i) La déduplication retire $\sim 32\%$ du pool : les chiffres absolus ne transfèrent qu'*approximativement* au jeu complet ($\sim 67k$). (ii) La segmentation SSE par `biotite.annotate_sse` n'est pas du vrai DSSP ; lors du ré-échantillonnage en sous-structures, l'assignation SSE de la protéine entière est sous-ensemblée, pas recalculée. (iii) Le 3Di vient de Foldseek — *or Foldseek est aussi notre métrique de validation*. Le 3Di est une entrée ; le clustering Foldseek est la cible. Ce n'est pas une fuite directe

mais c'est une dépendance à clarifier, d'où le drapeau `-no-3di`.

3 Architecture

3.1 Le PCC et la collation

`custom_collate` (`train.py`) empile B protéines en un seul graphe : décalage des indices de nœuds/SSE, `batch_idx_0` (nœud→protéine), `batch_idx_2` (SSE→protéine), `sse_map_0` (nœud→SSE). Les features de rang 2 sont concaténées en `rank2_features` ($S, 12$). Les protéines dont le graphe n'a pas exactement $k = 16$ voisins sont rejetées.

3.2 Encodeur invariant asymétrique (baseline)

`asymmetric_topotein.py` : passage de messages *dirigé*, *asymétrique*, *hiérarchique* par attention, sur scalaires uniquement. Il garde `rank1['distance'] = \|v\|` mais *jette* le vecteur `rank1['vector']` : toute orientation est perdue avant la couche 1 (d'où « version faible de topotein »). Une couche enchaîne quatre sous-étapes :

1. **Arêtes** $\leftarrow$ **nœuds source** (Rang0→Rang1) : attention $q_{\text{edge}} \cdot k_{\text{src}} / \sqrt{d}$.
2. **SSE** $\leftarrow$ **nœuds membres** (Rang0→Rang2), agrégation softmax par SSE.
3. **Résidus** $\leftarrow$ **arêtes entrantes** (Rang1→Rang0).
4. **Global** $\leftarrow$ **pooling nœuds** + **SSE** (mise à jour de h_3).

3.3 Mitigations « anti-raccourci »

Trois correctifs (sans nouveaux paramètres, donc rétro-compatibles avec les checkpoints) sont activés par l'entrée d'entraînement : `edge_attn_softmax=True` (vraie normalisation softmax des arêtes sortantes par nœud, au lieu d'un sigmoïde qui sature en moyenne non pondérée) ; `dist_bias_gamma>0` (biais géométrique $-\gamma d_{ij}$ ajouté aux logits d'attention, ancre l'attention sur la proximité 3D) ; `detach_h3=True` (détache les features poolées qui alimentent h_3 pour casser le chemin de gradient « raccourci global »). S'y ajoutent les drapeaux d'entrée `-no-positional` (tue le signal d'index de séquence → raccourci d'attention séquentielle), `-no-residue` (compose-shortcut) et `-no-3di` (raccourci « benchmark » Foldseek). Chaque drapeau met à zéro le canal correspondant sans changer les dimensions.

3.4 Encodeur équivariant (GVP / GCP)

`equivariant_topotein.py` porte un couple (**scalaire**, **vecteur**) par cellule et met à jour les deux via des perceptrons géométriques sur la même topologie. L'axe décisif est la *scalarisation* — comment l'information directionnelle entre dans les canaux scalaires (invariants) :

- `scalarize='norm'` (**GVP**, Jing et al. 2021) : un canal vecteur devient sa norme $\|v\|$ — une seule grandeur, direction jetée. Invariant sous tout $O(3)$, *réflexion comprise* : **ne distingue pas une structure de son image miroir** (aveugle à la chiralité). Rung « baseline équivariant léger ».
- `scalarize='frame'` (**GCP**, défaut) : chaque canal vecteur est *projeté sur un repère local complet* $F = (e_1, e_2, e_3)$ donnant trois scalaires *signés* $(v \cdot e_1, v \cdot e_2, v \cdot e_3)$, direction préservée. Comme $e_3 = e_1 \times e_2$ (pseudovecteur qui change de signe par réflexion), la projection est **sensible à la chiralité** : le modèle est $SE(3)$ -équivariant *et* chiral.

Le repère local par résidu (`build_node_frames`) est construit à partir de trois $C\alpha$ consécutifs (e_1 = tangente de chaîne, e_2 = normale dans le plan par Gram-Schmidt, $e_3 = e_1 \times e_2$) : invariant par translation, équivariant par rotation ($F \rightarrow FR$). Le GCP (`class GCP`) applique un mélange de canaux *sans biais* (équivariance), des grilles invariantes (*vector gate*), et une agrégation à poids d'attention invariants. Les garanties (invariance de lecture sous $SE(3)$, équivariance des canaux vecteurs, changement de lecture sous réflexion en mode **frame**) sont vérifiées par le self-test `__main__`.

3.5 Encodage de distance RBF

`rbf.py` remplace l'encodage sinusoïdal (mal adapté : sur 0–20 Å avec base 10 000 sur 16 dims, ~la moitié des bandes sont mortes) par des bases gaussiennes type SchNet : $e_k(d) = \exp(-(d-\mu_k)^2/2\sigma^2)$, centres μ_k régulièrement espacés sur $[3.5, 20]$ Å, $\sigma = \text{pas}$. Calibré sur la distribution empirique des distances de contact (p1≈3.7 Å, médiane≈7.5 Å, p99≈20 Å). Le scalaire de distance brute est conservé *en plus* de l'expansion.

3.6 Lecture : représentation vs sortie normalisée

Les deux encodeurs exposent `forward(features, return_repr=True)` qui renvoie h — la lecture poolée *avant* L2-normalisation. La sortie « standard » est `normalize(h)`. Distinction cruciale (§4.3) : la tête de projection contrastive consomme h , mais l'embedding *d'évaluation* est `normalize(h)` — la tête est jetée à l'inférence.

4 Objectif d'entraînement

4.1 InfoNCE / NT-Xent (`contrastive_engine.py`)

Pour un lot de B protéines en deux vues z_i, z_j normalisées, $z = [z_i; z_j] \in \mathbb{R}^{2B \times d}$, similarités $S = zz^\top / \tau$. Perte InfoNCE standard = entropie croisée où, pour chaque ligne, le positif est la vue appariée et les négatifs sont les $2B - 2$ autres. Option *hard-negative* (Robinson et al. 2021, `hard_neg_beta>0`) : repondère les négatifs vers les plus durs via des poids $\propto e^{\beta S}$ en log-espace.

4.2 SupCon et soft-SupCon

`supervised_ntxent_loss` : SupCon sur labels de cluster Foldseek (positifs = même cluster). `soft_supcon_loss` : positifs *pondérés par le TM-score réel* (cache pré-calculé) plutôt que binaires — transforme l'objectif « collapser chaque cluster en un point » en « placer les protéines à des distances proportionnelles à leur distance structurale ».

Ces deux objectifs supervisés utilisent les labels Foldseek / les TM-scores comme cible d'entraînement. C'est la fuite identifiée et retirée : entraîner sur Foldseek puis évaluer contre Foldseek/TM gonfle artificiellement les métriques. Le pipeline actuel (MoCo) est *non supervisé* précisément pour l'éliminer.

4.3 MoCo-v2 — l'objectif courant

`moco.py` (`plan_implementation_3`). Deux encodeurs : *online* f_q (entraîné) et *momentum* f_k (EMA, sans gradient), $\theta_k \leftarrow m\theta_k + (1-m)\theta_q$. Tête de projection g (MLP $d \rightarrow d \rightarrow d$, SiLU + L2-norm), version online et momentum. File FIFO de K clés négatives. Pour une protéine : $\text{vue1} \rightarrow q = g(f_q(\text{vue1}))$; $\text{vue2} \rightarrow k^+ = g_k(f_k(\text{vue2}))$ (no-grad). Perte :

$$L = -\log \frac{\exp(q \cdot k^+ / \tau)}{\exp(q \cdot k^+ / \tau) + \sum_{k^- \in \text{file}} \exp(q \cdot k^- / \tau)}.$$

À chaque step : mise à jour EMA, enfilage de k^+ , défilement des plus anciennes. La file est initialisée par des clés aléatoires normalisées (K négatifs dès le step 0). Pas de ShuffleBN (LayerNorm/GVP-norm, pas de BatchNorm). **Hyperparamètres** (défauts) : $K = 8192$, $m = 0.99$, $\tau = 0.2$ (le $\tau = 0.1$ du run 1 favorisait le collapse), tête jetée à l'éval. Justification (post-mortem run 1) : la file découple le nombre de négatifs de la (petite) taille de batch imposée par MPS, et remplace le hard-neg mining (retiré car il créait des faux négatifs et du collapse).

4.4 Augmentation par sous-structures (substructure.py)

Remplace l'augmentation triviale (jitter/crop/masquage $\rightarrow$ deux copies quasi identiques $\rightarrow$ loss $\sim 10^{-5}$, rien d'appris). La paire positive = **deux sous-structures connexes distinctes de la même protéine** (Hermosilla & Ropinski, arXiv:2205.15675). Une sous-structure est un *re-lifting CA-only* restreint à un sous-ensemble S de résidus : on sous-ensemble les features par résidu et on recalcule la géométrie (graphe kNN rang-1, ACP rang-2, globaux rang-3) sur `ca_coords[S]` avec les mêmes formules que le lifter. Mode par défaut `contiguous` (segment de chaîne, préserve la localité). Taille = fraction $f \sim U(0.5, 0.8)$ des résidus (hyperparamètre critique, « sweet spot » InfoMin : trop petit $\rightarrow$ les vues ne partagent plus le fold ; trop grand $\rightarrow$ vues quasi identiques). Plancher de 17 résidus (kNN $k = 16$ valide). La collate MoCo (`moco_collate`) garde les deux vues *séparées* (vue1 $\rightarrow f_q$, vue2 $\rightarrow f_k$).

5 Boucle d'entraînement et ingénierie système

`train_contrastive.py`. Pensée pour Apple Silicon (MPS, 16 Go), où le coût mémoire/temps est le facteur limitant.

- **Batching par budget de résidus** (`ResidueBudgetSampler`) plutôt que par compte fixe : empile jusqu'à `max_residues` résidus (défaut ~ 1800 pour le modèle équivariant lourd, 3500 pour l'asymétrique), avec `batch_size` comme plafond dur. Évite qu'un lot de grosses protéines fasse exploser l'allocateur MPS. (Le `HardNegativeBatchSampler` groupant par longueur/composition SSE existe encore mais est désactivé sous MoCo.)
- **Bucket padding** (`pad_to_buckets`) : complète les shapes à un petit nombre de tailles fixes pour que MPSGraph réutilise des kernels compilés (sinon recompilation par shape unique). Padding isolé des vraies protéines (nœuds/SSE/protéine factices), sortie tranchée à `real_B` avant la perte.
- **Gouverneur mémoire** (`MemoryGovernor`) : reclaim throttlé, reclaim hors-planning sous pression douce (`soft_gb`), et redémarrage à froid des workers + réduction dynamique du budget de résidus au-dessus du plafond dur (`hard_gb`), pour rester sous la falaise de swap à 16 Go.
- **Optimiseur** : AdamW (fused si dispo), lr 10^{-4} , weight decay 10^{-5} ; scheduler `ReduceLROnPlateau` (facteur 0.5, patience 2). Clipping : `clip_grad_value_` sur MPS (le `clip_grad_norm_` provoque des syncs CPU coûteuses).
- **Checkpointing** : à chaque epoch, `_last`, `_epoch{NNN}` (jamais écrasé) et `_best` (déclenché par meilleure val-loss *ou* meilleure ARI — car changer d'objectif change l'échelle de la loss).
- **Détection de collapse** en cours d'entraînement (tous les 10 steps) : alerte si `mean_cos > 0.9` ou `emb_std < 0.02`.

6 Métriques et évaluation

`scripts/analysis/epoch_eval.py`, calculé sur les embeddings de *validation* à chaque epoch (l'embedding = `normalize(h)` de la vue 1, tête jetée).

- **TM-rho** (cible principale) : Spearman ρ (distance cosinus, TM-score) sur les paires en cache. **Plus négatif = meilleur** (forte distance cosinus quand TM faible). C'est la métrique « le plongement encode-t-il une métrique structurale continue ? ».
- **TM-recall** : fraction de paires inter-structures (TM > 0.5) placées dans la bande cosinus proche (< 0.45) — récupération d'homologie.
- **ARI / HDBSCAN ARI & NMI**, plus V-measure, homogénéité, complétude, Fowlkes–Mallows, et *fragmentation/fusion* (un cluster Foldseek éclaté en combien de clusters appris, et l'inverse — exactement les diagnostics demandés par C.). **perm_ari** : ARI sous permutation des labels (plancher calibré ; ARI réel – perm_ari = apport des labels).

- **Santé / anti-collapse** (§6.1) : *effective rank* (Roy–Vetterli, $\sim D$ bon, ~ 1 collapse dimensionnel), `emb_std`, `mean_cos`, et *uniformity* (Wang & Isola).

7 Résultats réels

7.1 Le balayage d'ablations (encodeur asymétrique)

14 configurations, 6 epochs chacune (`checkpoints/sweep/`). **Baseline « features brutes + init. aléatoire » : $\text{TM-rho} = -0.48$.** Une config « bat la baseline » seulement si son meilleur TM-rho est plus négatif que -0.48 .

Table 2: Extrait du balayage, trié par meilleur TM-rho (plus négatif = mieux). Colonnes : meilleur ρ / epoch, `recall`, `min_std` et `max_cos` (santé collapse).

Run	best ρ	ep	recall	min_std	max_cos	note
E_no_hardneg	-0.473	2	0.688	0.064	0.404	sain, ne bat pas -0.48
E_nohn_jitter1.0	-0.471	1	0.668	0.065	0.382	sain
geomonly_nohn	-0.468	1	0.704	0.068	0.315	géométrie seule
feat_geomonly	-0.455	3	0.690	0.028	0.861	collapse partiel
E_nohn_tau0.05	-0.449	1	0.753	0.048	0.652	
base	-0.437	2	0.646	0.023	0.903	« bat » mais collapse
C_jitter0.6	-0.396	2	0.638	0.025	0.888	

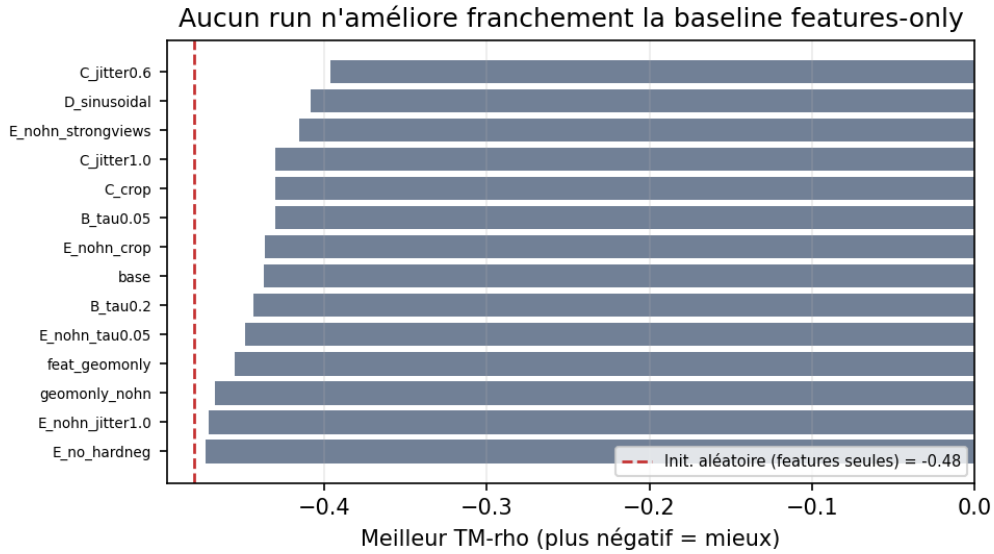


Figure 2: Meilleur TM-rho par run *vs* baseline init. aléatoire (-0.48 , pointillés rouges). Les rares runs qui franchissent la ligne le font au prix d'un collapse (`max_cos` ≈ 0.9). Le meilleur run sain (-0.473) reste *en deçà* de la baseline.

7.2 Le run MoCo + sous-structures

Run le plus récent (`checkpoints/moco_geom/`, encodeur *asymétrique*, 4 epochs). Loss InfoNCE MoCo ≈ 5.4 – 5.8 (chance = $\ln(K+1) = 9.01$: la tâche est bien non triviale, contrairement au run 1). ARI val : $0.10 \rightarrow 0.13 \rightarrow 0.15 \rightarrow 0.13$ (Fig. 3). Les métriques structurales (TM-rho, eff. rank...) ne sont pas encore loguées pour ce run.

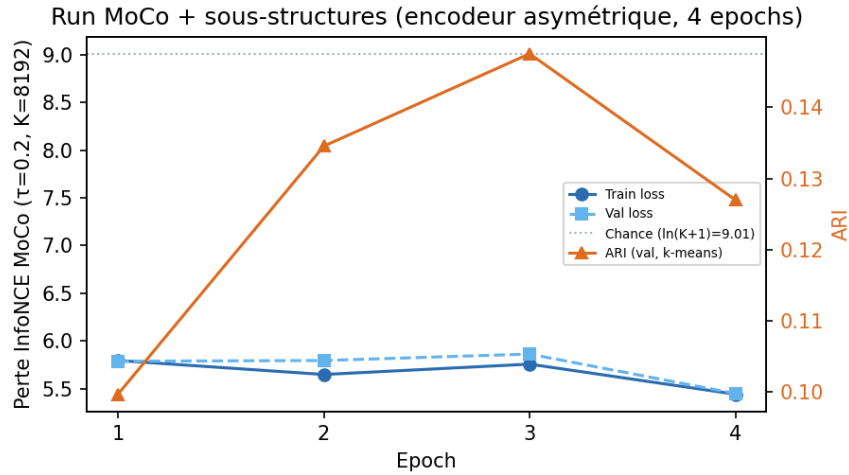


Figure 3: Run MoCo + sous-structures : la loss descend bien sous la chance (tâche non triviale, porte A franchie), l'ARI augmente puis fléchit à l'époch 4.

7.3 Évaluation sur la sous-base corrigée

checkpoints/epoch_eval.csv (split corrigé, 4 004 protéines, ~7 059 paires TM) :

epoch	n_clusters	singleton_frac	HDBSCAN ARI	HDBSCAN NMI	TM-rho	TM-recall
0 (init. aléatoire)	737	0.244	0.082	0.778	-0.584	1.000
1 (entraîné)	989	0.289	0.061	0.770	-0.501	0.690

Le constat dur, à dire tel quel. Sur le split corrigé, après une epoch, TM-rho passe de -0.584 (init.) à -0.501 : *moins négatif*, donc **l'entraînement dégrade la métrique cible par rapport à l'init. aléatoire**. HDBSCAN ARI baisse ($0.082 \rightarrow 0.061$), TM-recall chute ($1.00 \rightarrow 0.69$), et le modèle sur-fragmente ($737 \rightarrow 989$ clusters). Conjugué au balayage (aucun run sain ne bat -0.48), **le signal est cohérent : une fois la fuite retirée, le modèle entraîné n'apporte rien — voire nuit — au-dessus des features brutes**. C'est exactement la question ouverte de `reu_celine.md` : « problème de pipeline d'apprentissage *ou* problème d'expressivité du modèle ? ».

Nuance importante : le tableau ci-dessus ne couvre qu'une epoch entraînée. Affirmer « l'entraînement nuit » sur la base d'un seul epoch est prématuré ; il faut le run complet (et les TM-rho du run MoCo) avant de conclure. C'est précisément le point à trancher.

8 Angles morts et questions probables du superviseur

Section délibérément critique (rôle de mentor).

- Pipeline ou expressivité ? Le test décisif manque.** Tu n'as pas encore isolé les deux hypothèses. Test minimal : (a) le *même* encodeur entraîné en *supervisé* (SupCon sur Foldseek) atteint-il un bon TM-rho ? Si oui $\Rightarrow$ l'architecture est expressive, le problème est l'objectif non supervisé/les paires. Si non $\Rightarrow$ problème d'expressivité ou de features. (b) Une *sonde linéaire* sur les features brutes (3Di + ACP) prédit-elle déjà le TM-score aussi bien que l'embedding ? Si oui, le réseau n'ajoute rien.
- La baseline à -0.48 est probablement dominée par le 3Di de Foldseek.** Le 3Di est un alphabet *structural* appris par Foldseek ; il porte déjà l'essentiel du signal que tu mesures avec une métrique Foldseek. Question : quelle est la TM-rho des features *purement géométriques*

(sans 3Di, -no-3di) à l'init ? Si elle s'effondre, ta baseline « features » est en réalité « Foldseek réemballé ».

3. **Le run MoCo le plus récent tourne sur l'encodeur *asymétrique* (invariant), pas l'équivariant.** Or toute la justification GCP/chiralité concerne l'équivariant. Tu présentes une architecture équivariante mais tes derniers résultats viennent de la version qui *jette* les vecteurs. À clarifier : lequel est « le modèle » ?
4. **Sélection de modèle sur une seule epoch / métrique instable.** L'ARI du run MoCo fléchit déjà à l'epoch 4 ; le balayage montre que le meilleur epoch est souvent 1–2 puis ça se dégrade — signe d'un objectif mal aligné sur la cible (la val-loss s'améliore pendant que TM-rho empire). Le checkpoint « best » par val-loss est trompeur ici.
5. **Interprétation du signe de TM-rho : un doc interne se contredit.** METRICS_SUMMARY.md (11 juin) écrit « trending toward zero = improving », ce qui est **faux** : epoch_eval.py et le balayage définissent « plus négatif = mieux ». Vérifie quelle convention tu présentes — une erreur de signe inverserait toutes tes conclusions.
6. $\tau = 0.2$, $K = 8192$, $f \in [0.5, 0.8]$ **ne sont pas balayés sous MoCo.** Ces hyperparamètres viennent du plan, pas d'une recherche. La taille de sous-structure f est annoncée comme « le nouveau point de défaillance principal » — or elle n'est pas encore balayée.
7. **Pas de témoin de clustering aléatoire.** C. a demandé un témoin (clusters aléatoires à tailles préservées). Il valide surtout le supervisé, mais permettrait de calibrer l'ARI. Absent à date.
8. **Repli prévu mais non testé.** Le plan prévoit VICReg/Barlow Twins (anti-collapse par construction) ou un objectif de denoising (GeoSSL) si le contrastif échoue. Vu les résultats, c'est peut-être le moment de l'envisager — à proposer plutôt que d'empiler des epochs MoCo.

Références

- Hermosilla, Ropinski. *Contrastive Representation Learning for 3D Protein Structures*, 2022. arXiv:2205.15675.
- He et al. *Momentum Contrast (MoCo)*, 2020. arXiv:1911.05722.
- Chen et al. *SimCLR*, 2020. arXiv:2002.05709.
- Jing et al. *Learning from Protein Structure with GVP*, 2021 ; *Dimensional Collapse*, 2022. arXiv:2110.09348.
- Robinson et al. *Contrastive Learning with Hard Negative Samples*, 2021. arXiv:2010.04592.
- Wang, Isola. *Alignment and Uniformity*, 2020. arXiv:2005.10242.
- Bardes et al. *VICReg*, 2022. arXiv:2105.04906 ; Zbontar et al. *Barlow Twins*, 2021. arXiv:2103.03230.
- Zhang et al. *GearNet / Multiview Contrast*, ICLR 2023. arXiv:2203.06125.
- van Kempen et al. *Foldseek / 3Di*, Nat. Biotechnol. 2024.